

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Designing the Flutter Web Client-Side Interface for Your Own Project

Flutter Web Client-Side Design

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 6
Output	Flutter UI-flow diagram, responsive wireframes/prototype, widget and validation plan, and design review
Maximum Score	80 points

Student / Project Information

Block: <u>A-1</u>	Date: <u>September 20, 2026</u>
System Title: Seaweed (Guso) Quality Classification Using Image Processing	Instructor: Jean Y. Villasin, MIT
Members: Zydane Angelo Adanza, Timothy Josua Tavera, Jherick Peñalosa	

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 6

1. Activity Alignment and Purpose

Topic Learning Outcome: Design the Flutter Web user interface, navigation flow, validation plan, and responsive behavior for the semester system.

Course Outcome Alignment: Supports semester-project client-side design by converting approved requirements and use cases into a usable, accessible, responsive, and implementation-ready Flutter Web interface plan.

Syllabus Assessment: Flutter UI flow/wireframe; design review; online interface-design exercise.

Purpose of the Activity

Activity 5 established the requirements baseline and system-specific use case diagram. In Activity 6, your team will translate those approved user goals and requirements into the actual client-side experience of your semester system. You will plan the screen sequence, navigation and routes, responsive layouts, reusable Flutter widgets/components, forms and validation behavior, accessibility and usability decisions, and the loading, empty, success, and error states that users will encounter. The final UI-flow and responsive wireframes will serve as the implementation blueprint for the Flutter Web client in the succeeding development activities.

Important distinction: A wireframe defines structure, hierarchy, content placement, and interaction flow; a prototype simulates interaction; the final Flutter interface is the implemented product. Client-side validation improves usability but does not replace validation and authorization in the REST API.

2. Intended Learning Objectives

- Translate the approved SRS, system scope, user roles, and use cases into a coherent Flutter Web screen flow. • Identify appropriate widget-based UI structures and distinguish presentational/static components from components that require local or shared state.
- Plan navigation, named routes or route patterns, protected screens, back/exit behavior, and user-role access to major screens.
- Create responsive wireframes that adapt intentionally across representative mobile, tablet, and desktop web widths. • Design reusable interface components and a consistent visual system for forms, cards, navigation, lists, dialogs, feedback, and common actions.
- Define client-side form validation and loading, empty, success, and error states for important user workflows. • Review the design for accessibility, usability, requirement traceability, and implementation readiness before Flutter coding begins.

3. Activity Specifications

Activity Item	Requirement
---------------	-------------

Mode	Existing semester-project group; every member must contribute to flow analysis, wireframing, review, and oral explanation.
Suggested Time	Approximately 3 laboratory hours plus the scheduled design review and online interface-design exercise.
Primary Output	One system-specific Flutter UI-flow diagram and a responsive wireframe/prototype set based on the approved project requirements.
Supporting Output	Screen inventory; widget/state plan; route map; reusable-component plan; form-validation plan; UI-state matrix; accessibility/usability checklist; design-review revision log.
Recommended Tools	Figma, Penpot, diagrams.net/Draw.io, Canva, PowerPoint, Flutter/Dart for optional clickable prototype work, or an equivalent instructor-approved design tool.
Design Review	5–7 minutes per group followed by 2–3 minutes of questions. Every member must be able to explain assigned screens, routes, responsive decisions, and interface states.

4. Pre-Laboratory Requirements

- Approved or revised SRS, user roles/permissions, scope, requirements-to-feature matrix, and system-specific UML use case diagram from Laboratory Activity No. 5.
- Prescribed Flutter Web-REST API-PostgreSQL technology-stack blueprint from Laboratory Activity No. 3 and current architecture diagram from Laboratory Activity No. 2.
- Selected development methodology and semester milestones from Laboratory Activity No. 4.
- List of Must/High-priority features and the user workflows that must be demonstrated during the semester project.
- Any existing branding, content, icons, forms, data samples, or interface references relevant to the team's actual system.
- A shared design workspace or project repository where the UI-flow, wireframes, screenshots, and review notes can be versioned.

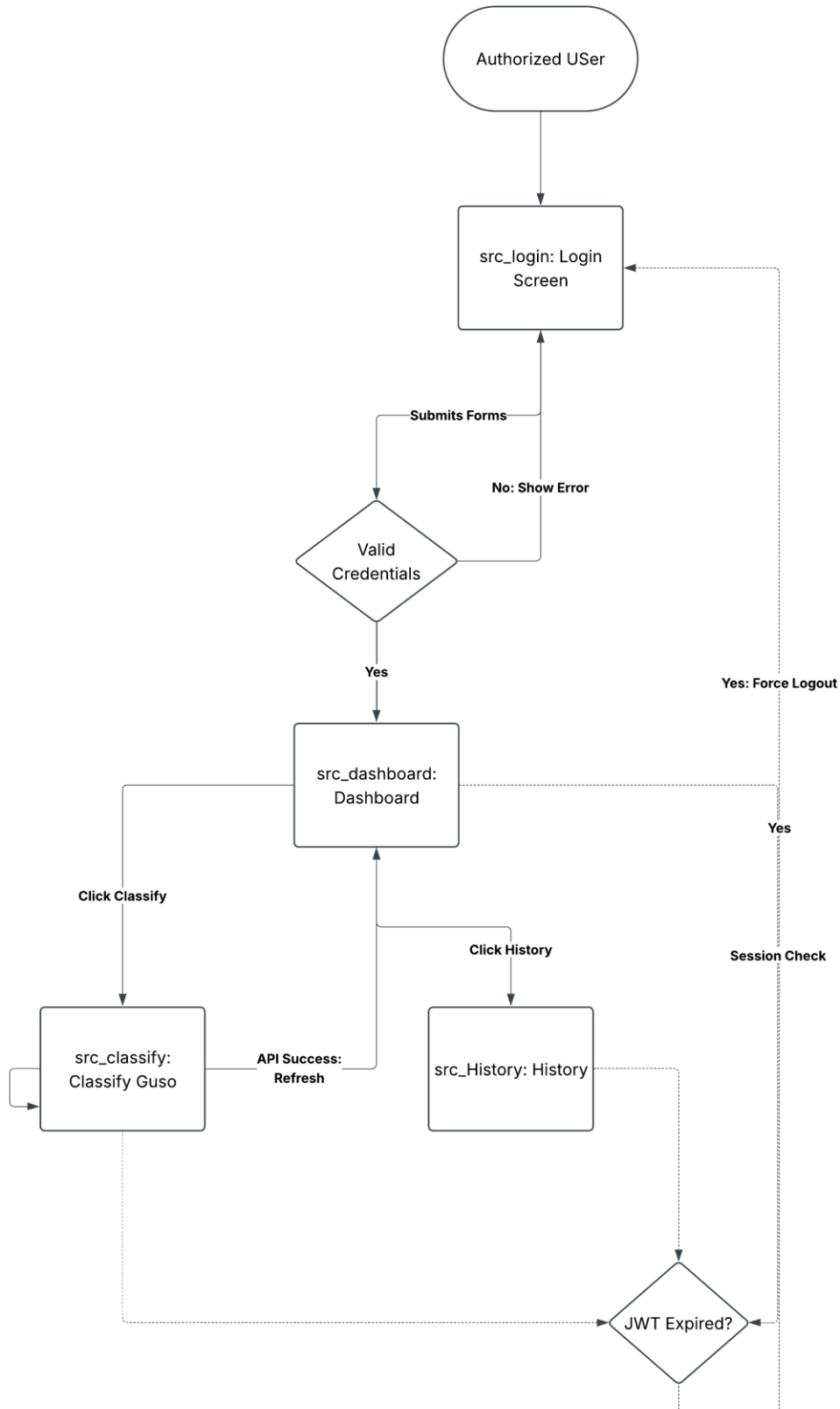
5. Part A – Translate Requirements and Use Cases into the UI Flow

Start with the approved requirements and use case diagram—not with visual styling. For each important user goal, identify where the user enters the interface, which screens or overlays are required, where decisions or validation occur, and what successful completion looks like.

Use Case / Req. ID	Actor / Role	Entry Point	Required Screen Sequence	Decision / Validation Point	Expected Destination
UC/US-01 / FR-01, 02	Farmer / Seller	/register	Registration Form → OTP Modal	Check required fields, password match, and valid OTP code format	/login (on success)
UC/US-02 / FR-03	Farmer / Seller	/login	Login Screen	Validate non-empty credentials before submission.	/dashboard
UC/US-03, 04 / FR-04, 05	Farmer / Seller	/dashboard	Dashboard → Classification Screen	Client checks if the selected file is a valid image (.jpg, .png) under 5MB.	/classification-result

UC/US-06 / FR-08	Farmer / Seller	/dashboard	Dashboard → History Screen	N/A (read-only view)	/record-detail
---------------------	-----------------	------------	----------------------------	----------------------	----------------

System: Guso Quality Classification Using Image Processing | Role: Farmer / Seller |
Version: 1.0



Traceability Rule

Every Must/High-priority user-facing requirement should lead to one or more identifiable screens, actions, or interface states. If the wireframes show a major feature that is not supported by the approved requirements, flag it for requirements review before implementation.

6. Part B – Screen Inventory and Widget-Based UI Planning

Create a complete screen inventory before polishing individual wireframes. Use actual project screen names. The “widget/state plan” is a design annotation—not a requirement to write the final Dart code during this activity.

Screen ID / Name	Purpose / Main User	Major UI Sections / Candidate Flutter Widgets	Reusable Component Candidates	State Needed	Data / API Dependency
Screen ID / Name	Purpose / Main User	Major UI Sections / Candidate Flutter Widgets	Reusable Component Candidates	State Needed	Data / API Dependency
scr_login	Authenticate Farmer/Seller	Scaffold, Form, TextFormField, blue gradient primary button	Minimal Primary Button, Form Input Field	Local form state (obscure password), Riverpod AuthState	FastAPI /api/auth/login
scr_dashboard	Central hub for Farmer/Seller	Scaffold, GridView for quick actions, ListView for recent history	Activity Card, Custom App Bar	Riverpod UserState	FastAPI /api/users/me
scr_classify	Upload and grade Guso image	Column, Image.file preview, ElevatedButton	Image Picker Area, Status Feedback	Local file state, Riverpod ClassificationState (loading/success)	FastAPI /api/classifications
scr_history	View past classification records	ListView.builder, ListTile	History Record Row	Riverpod HistoryState	FastAPI /api/classifications/history

Widget and State Classification Guide

Design Question	Likely UI Planning Direction	Examples / Notes
-----------------	------------------------------	------------------

Does the component mainly render information from inputs and have no local mutable interaction state?	Plan it as a presentational/reusable widget; it may later be implemented as a StatelessWidget or equivalent composition.	Header, label, summary card, icon-text row, static empty-state panel.
Does the component own temporary local UI state that changes during interaction?	Identify the local state and event that changes it; a StatefulWidget may be appropriate during implementation.	Password visibility, tab selection, expansion, local form interaction.
Is the state shared across screens or tied to asynchronous application data?	Do not force it into one StatefulWidget. Identify the shared/application-state boundary and planned state-management approach.	Authenticated user, loaded records, filters used across screens, async request state.
Can the same structure or behavior appear in several screens?	Extract it as a reusable component with clearly defined inputs, callbacks, and states.	Buttons, form fields, cards, status chips, confirmation dialogs, table/list rows.

Possible Flutter building blocks to annotate when appropriate: Scaffold, AppBar, NavigationRail/Drawer, Row, Column, Wrap, GridView, ListView, Card, Form, TextFormField, DropdownButton/FormField, buttons, dialogs, LayoutBuilder, MediaQuery, and project-specific reusable widgets.

7. Part C – Navigation and Routing Plan

Plan how users move between screens and what should happen when they use browser navigation, direct URLs, sign in/out, role-restricted routes, or invalid routes. The final route structure must match the approved user roles and scope.

Route / Path	Screen / View	Allowed Role(s)	Entered From	Primary Exit / Back Behavior	Guard / Condition
/login	Login Screen	Public	Browser entry	Exit application	Redirects to /dashboard if already authenticated.
/dashboard	Dashboard	Farmer, Seller	/login	Prompts logout confirmation	Requires valid JWT token in Riverpod state.

/classify	Classification Screen	Farmer, Seller	/dashboard	Returns to /dashboard	Requires valid JWT token
/history	History Screen	Farmer, Seller	/dashboard	Returns to /dashboard	Requires valid JWT token

- Show the normal route sequence for at least one end-to-end critical workflow.
- Mark routes that require authentication, a specific role, ownership of a record, or another access condition.
- Define what the interface should do for an unknown/invalid route and after session expiration.
- Design navigation so that browser back/forward behavior and direct URLs do not create confusing or unauthorized states.

8. Part D – Responsive Layout and Wireframe Planning

Flutter Web must respond to available space rather than merely shrink a desktop canvas. Use representative widths to test the layout. The exact breakpoints may differ by project, but the team must demonstrate intentional adaptation.

Screen / Module	Narrow / Mobile Behavior	Medium / Tablet Behavior	Wide / Desktop Behavior	Overflow / Long-Content Strategy
Screen / Module	Narrow / Mobile Behavior	Medium/Tablet Behavior	Wide / Desktop Behavior	Overflow/Long-Content Strategy
Dashboard	1-column ListView, bottom navigation bar, hamburger menu.	2-column GridView for actions, side drawer for navigation.	Persistent NavigationRail on the left, 3-column grid, clean minimal layout.	Vertical scrolling using SingleChildScrollView.
Classification Screen	Image preview stacks vertically above the classification result and upload button.	Image preview and buttons centered with constrained maximum width.	Image preview on the left side, classification details and actions on the right side.	Scrollable column for smaller viewports to prevent pixel overflow.
History Screen	Standard ListTile rows with compressed date and grade text.	Expanded table or grid view showing Guso thumbnails.	Full data table with columns for Date, Thumbnail, Grade, and Download Action.	Standard vertical scrolling; pagination if records exceed 50 items.

Minimum Responsive Wireframe Requirement

Produce wireframes for all major screens at the team's primary desktop/web width. In addition, show at least three critical screens at three representative widths (for example, approximately 390 px, 768 px, and 1440 px) so the reviewer can see how navigation, columns, cards, forms, tables/lists, spacing, and actions adapt. Do not rely on tiny text or uncontrolled horizontal overflow as the responsive solution.

9. Part E – Reusable Components and Visual Consistency Plan

Identify the recurring pieces of the interface before implementation. Reusable components should reduce duplicated UI logic while preserving consistent behavior and visual language across the system.

Component / Pattern	Purpose	Used in Screens	Inputs / Variants	Interactive State / Callback	Accessibility / Usability Note
Primary action button	Standardize main actions with a minimal, blue gradient visual theme.	Login, Classify	Label (String), isLoading (Bool)	onPressed, shows CircularProgressIndicator if loading	High contrast white text on blue gradient; large touch target.
Form field pattern	Standardize text entry.	Login, Register	Hint text, obscure text, validator function	Focus state, error text display	Clear labels; visible focus border.
Navigation item	Display a single classification record cleanly.	Dashboard, History	Record object (Grade, Date, Image URL)	onTap to view full details	Readable font hierarchy (Grade bolded).
Record card / row	Display a single classification record cleanly.	Dashboard, History	Record object (Grade, Date, Image URL)	nTap to view full details	Readable font hierarchy (Grade bolded).
Status / feedback component	Show system alerts or ML confidence scores.	Classify, Login	Message string, status type (error/success)	Auto-dismiss timer or manual close IconButton	Do not rely on color alone; use icons for errors.
Project-specific component	Display the selected Guso image preview.	Classify	File object, isLoading boolean	onTap to change image	semanticLabel describing the uploaded image.

Create a simple interface consistency guide covering: typography hierarchy, spacing system, primary/secondary actions, form-field states, color usage, cards/containers, icon conventions, navigation patterns, confirmation behavior, and feedback messages.

10. Part F – Forms and Input Validation Plan

For each important form, define both the visible interaction and the validation behavior. The message shown to the user should explain what must be corrected. Client-side checks are for immediate feedback; the REST API must still validate submitted data and enforce permissions.

Form / Field	Input / Control	Client-Side Rule	When Error Appears	Error / Helper Message	Submit / API Behavior
Login Password	TextFormField	Cannot be empty, minimum 8 characters	On field exit and on submit.	"Password is required / must be at least 8 chars"	Submits to API; shows loading indicator.
Image Upload	Custom File Picker	Must be .jpg or .png, file size < 5MB	Immediately upon file selection.	"Invalid file type. Please upload a JPG or PNG under 5MB."	Button is disabled until a valid image is in local state.

- Consider required fields, length, format, numeric/date range, cross-field rules, file type/size, duplicate input, and conditional fields when relevant.
- Show disabled/submitting states so users cannot accidentally submit the same action repeatedly.
- Decide where validation is inline, on field exit, on submit, or a combination—then apply the pattern consistently.

11. Part G – Accessibility and Usability Review

Review the wireframes from the perspective of a user who may navigate by keyboard, use assistive technology, have reduced vision, or encounter the system under time pressure or unreliable connectivity.

Review Area	Required Design Evidence	Status / Revision Note
Labels and instructions	Controls and inputs have visible, understandable labels; placeholder text is not the only label.	Pass. All TextFields have permanent top labels.
Keyboard and focus	Interactive controls have a logical focus order and visible focus state; key workflows do not require a mouse.	Pass. White text on blue gradients meets contrast ratios; error states use icons + text.
Contrast and non-color cues	Text and controls remain understandable with sufficient contrast; status/error meaning is not conveyed by color alone.	Pass. Minimum 48x48 logical pixels for mobile targets.
Text and layout resilience	Important content remains readable when text is enlarged or the viewport becomes narrow.	Pass. Columns reflow to single-column vertical stacks on mobile viewports.
Touch/click targets	Buttons, icons, checkboxes, and menu items are large enough and spaced to prevent accidental activation.	Pass. Minimum 48x48 logical pixels for all primary actions and menu items.
Feedback and recovery	Errors identify the problem and recovery action; destructive actions use suitable confirmation/undo patterns.	Pass. ML API errors explicitly provide a "Retry" button rather than just a generic fail message
Semantics / meaningful content	Images/icons and page structure have meaningful labels/semantics where they communicate information.	Pass. All icons (e.g., History, Dashboard) have

		accompanying text labels.
Consistency and cognitive load	Navigation, actions, terminology, and form behavior stay consistent across screens.	Pass. The blue gradient primary action button is used identically across all forms.

12. Part H – Loading, Empty, Success, and Error-State Design

Do not design only the ideal “happy path.” Every major data-driven screen or submit action should have a planned state for waiting, no data, completion, and failure.

Screen / Action	Loading State	Empty / No-Data State	Success State	Error / Recovery State	Access / Offline Note
Classify Guso	Primary button turns into a loading spinner; UI locks.	"Upload a Guso image to begin classification."	Shows Grade, Confidence %, and a green checkmark.	"Classification failed. Check connection." + Retry Button.	Requires internet for MobileNetV3 inference.
History Screen	Skeleton loading rows replacing the list.	Minimal clean graphic: "No classification records found."	Renders the full list of records.	"Unable to load history." + Pull-to-refresh.	Cached records can be viewed offline if implemented.

State-Design Rule

Each state should answer four questions: What happened? What is the system doing now? What can the user do next? Is the action safe to retry? Loading indicators should not hide indefinitely, empty states should offer a meaningful next step when appropriate, and error states should preserve user work whenever possible.

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 6 **13. Part I – Construct the Flutter UI Flow and Responsive Wireframes**

Create one coherent design package that a teammate could use to begin Flutter implementation without guessing the basic navigation, screen hierarchy, responsive behavior, validation, and interface states.

No.	Required Element	What Must Be Shown
1	System identity	Official system title, group, design version/date, and the user role(s) represented in the flow.

2	Screen inventory	Every major user-facing screen required by the approved project scope, using stable screen IDs/names.
3	UI-flow diagram	Directional links between screens/overlays, with labels for important user actions, decisions, and success/error branches.
4	Navigation and routing	Planned route/path or route name, main navigation structure, role restrictions, and important entry/exit behavior.
5	Widget-based structure	Annotated sections/components showing suitable Flutter UI building blocks and the separation of reusable components.
6	State planning	Identify where local interaction state, shared/application state, or asynchronous request state is required.
7	Responsive behavior	Desktop/wide layout plus responsive adaptations; at least three critical screens demonstrated at three representative widths.
8	Forms and validation	Key inputs, controls, validation conditions, error feedback, submit behavior, and disabled/loading states.
9	Accessibility/usability	Labels, focus/keyboard behavior, readable hierarchy, contrast/non-color cues, target sizing, and recovery guidance.
10	System feedback states	Loading, empty, success, and error/retry states for important data screens and transactions.
11	Requirement traceability	Wireframe screens/actions are labeled or cross-referenced to the relevant use cases, requirements, or Must/High-priority features.
12	Design consistency	Common typography, spacing, component behavior, terminology, and action placement are applied consistently across the design.

Minimum UI-Flow Rule

The design package must contain at least one complete critical path from user entry or sign-in -> navigation -> feature screen -> form/detail action -> validation/request state -> success or recoverable error -> updated destination. The flow must be understandable even without opening the Flutter code.

Not Copy as Your Final Output

The example below shows the level of flow clarity expected. Replace every label, screen, state, action, and route with the actual design of your approved semester system.

Reference only: derive the actual screens, routes, states, and responsive behavior from your own SRS and use cases.

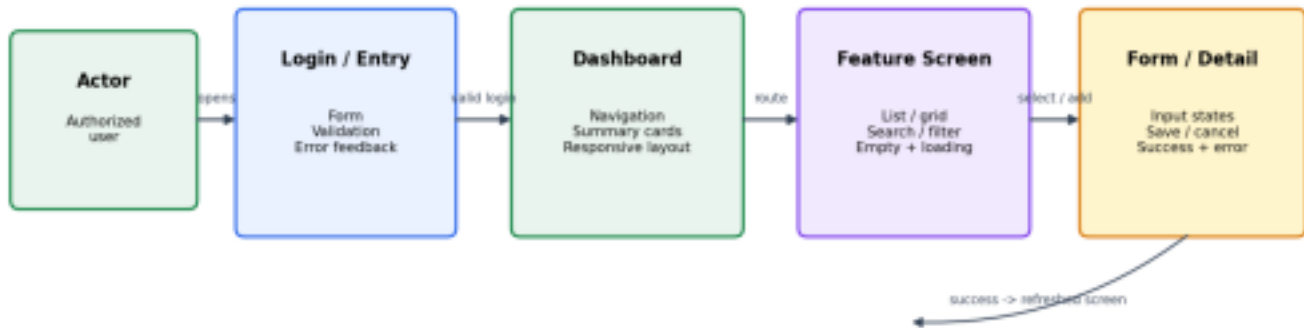


Figure 1. Reference client-side flow showing screens, navigation, validation, and interface states.

15. Recommended Flutter UI Design Annotations

Area	Possible Annotation	What It Communicates
Page shell	Scaffold / responsive shell	Overall page structure, navigation, content area, and global feedback region.
Navigation	NavigationRail / Drawer / menu / tabs	How primary destinations change by viewport and role.
Layout	Row / Column / Wrap / Grid / List	How content groups, wraps, stacks, scrolls, or changes column count.
Form	Form / TextFormField / dropdown / date input	Input grouping, validation feedback, submit behavior, and field states.
Reusable UI	ProjectCard / StatusChip / AppButton / FieldLabel	A project-specific reusable component and its variants.
Responsive logic	LayoutBuilder / MediaQuery / width class	Where structure or component presentation changes because of available space.
Async state	loading / empty / success / error	What the user sees before, during, and after a data request or mutation.

Phase 1 – Review the approved requirements (20 minutes)

Identify the Must/High-priority use cases, user roles, permissions, and major data interactions that require client-side screens.

Phase 2 – Build the screen inventory and UI flow (25 minutes)

List the screens, assign stable screen IDs, connect them using the actual user workflow, and identify decisions, guards, and important feedback states.

Phase 3 – Plan navigation, widgets, and reusable components (30 minutes)

Define the route structure, primary navigation pattern, major widget groups, reusable components, and state boundaries.

Phase 4 – Draft responsive wireframes (50 minutes)

Create the major desktop/web wireframes and the required responsive variants for at least three critical screens.

Phase 5 – Add validation and interface states (20 minutes)

Annotate forms, client-side validation, loading, empty, success, and error/recovery behavior.

Phase 6 – Accessibility and usability review (15 minutes)

Check keyboard/focus behavior, labels, contrast/non-color cues, layout resilience, target sizing, and user

recovery. **Phase 7 – Conduct peer/instructor design review (15 minutes)**

Use the review log below. Record defects, unclear flows, missing states, requirement mismatches, and responsive problems. **Phase 8 – Revise, export, and submit (5 minutes)**

Update the design version, export required files, commit the design artifacts, and submit the required links/files through google classroom.

17. Part J – Design Review and Revision Log

Use this log during the group, peer, or instructor design review. Do not silently fix major findings; record the problem and the revision decision so the team can explain how the design improved.

Issue ID	Reviewer Finding / Question	Affected Screen / Use Case	Priority	Revision Decision / Action	Status
REV-01	Large images from modern phone cameras will exceed the 5MB client validation limit.	scr_classify / Image Upload	High	Implement automatic client-side image compression in Flutter before validation and upload.	Resolved
REV-02	The UI flow diagram needs to clearly show how the system routes users with expired JWT tokens.	Global Routing / Diagram	Medium	Update the Draw.io UI flow diagram to include an explicit redirect loop from /dashboard back to /login when token expires.	Pending

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 6 18. Final Quality Checklist

Checklist A	Checklist B
☐ UI flow is derived from the approved SRS and use case diagram.	☐ Major screens are wireframed for the primary desktop/web width.
☐ Every Must/High-priority user-facing workflow maps to identifiable screens/actions.	☐ At least three critical screens show intentional mobile, tablet, and desktop adaptations.
☐ Screen names/IDs and route labels are consistent across the flow and wireframes.	☐ Layouts avoid unreadably small text and uncontrolled horizontal overflow.
☐ Protected or role-specific screens identify the relevant access condition.	☐ Keyboard/focus, labels, contrast/non-color cues, target size, and error recovery are reviewed.
☐ Reusable components are identified instead of duplicating the same pattern across screens.	☐ Terminology, buttons, spacing, navigation, and feedback patterns are consistent.
☐ Forms show required fields, validation behavior, error feedback, and submit/loading state.	☐ The design-review log records findings and the team's revision decisions.
☐ Loading, empty, success, and error/retry states are designed for important data workflows.	☐ Every member can explain assigned screens, routes, state behavior, and responsive choices without reading a script.

19. Submission Requirements

- Final Flutter UI-flow diagram in PDF or PNG format plus the editable source file.
- Responsive wireframe/prototype package showing all major screens and the required three-width variants for at least three critical screens; submit the editable file or accessible share link when the tool is web-based.
- Completed screen/widget inventory, route plan, reusable-component plan, validation plan, UI-state matrix, accessibility/usability review, and design-review revision log.
- Updated project repository or shared workspace under a folder such as docs/ui-design/ containing the current design version and exported review artifacts.
- Repository/share link and required files submitted through MasaoLMS using the naming convention IT110_LA6_SystemName.
- Complete the online interface-design exercise and incorporate instructor feedback that affects the project interface before implementation.

Academic Integrity and Competency Validation

The UI flow and wireframes must represent the team's own approved system, users, requirements, and design decisions. AI or design templates may be used to clarify concepts or accelerate drafting, but the submitted screen flow, labels, responsive choices, validation behavior, accessibility decisions, and state designs must be reviewed and understood by the team. The instructor may ask any member to trace a workflow or justify a screen/component decision without assistance.

F-CID-010 Rev.3, 03-21-2025 Competent • Service-oriented • Upright Page 11 of 13

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 6 20. Scoring Rubric – 80

Points

Partial credit may be awarded when evidence is present but incomplete, inconsistent with the approved requirements, technically weak, visually unclear, or not responsive. Design-review points may be reduced for unequal participation, excessive reading, or inability to explain the team's own interface decisions.

Criterion	Evidence for Full Credit	Points
Requirements-to-UI-flow traceability	The UI flow is complete for the approved scope; Must/High-priority use cases and requirements map clearly to screens, actions, decisions, and expected destinations.	10
Navigation and routing plan	Routes, navigation hierarchy, entry/exit behavior, role restrictions, protected screens, and important browser/back behavior are clear and internally consistent.	10
Responsive wireframes and layout design	Major screens are complete and readable; at least three critical screens demonstrate intentional adaptation across representative mobile, tablet, and desktop widths without uncontrolled overflow.	15
Widget structure and reusable components	The design shows sensible widget-based composition, identifies reusable UI patterns, and distinguishes local interaction state from broader/shared or asynchronous application state.	10
Forms, validation, and interface states	Important forms include usable validation and feedback; loading, empty, success, error/retry, disabled, and submit states are appropriately designed.	10
Accessibility and usability	The design addresses labels, focus/keyboard access, readable hierarchy, contrast/non-color cues, target sizing, consistency, user guidance, and recovery from errors.	8

Visual consistency and design documentation	Screen IDs, routes, terminology, typography, spacing, controls, annotations, and versioned design artifacts are clear, consistent, and implementation-ready.	7
Design review and oral defense	The group uses the UI flow/wireframes effectively, every member participates, revisions are documented, and the team can justify responsive, navigation, component, validation, and state decisions.	10

Raw Score	Equivalent	Checked by	Date
/ 80	_____	_____	_____

21. Post-Laboratory Reflection

Which user workflow was the most difficult to translate from the SRS/use case diagram into a clear screen flow, and why?

The Guso classification upload workflow. It was challenging to map out the transitional states (selecting an image, waiting for the FastAPI upload, waiting for MobileNetV3 inference) without confusing the user. We had to carefully separate the local image preview state from the asynchronous server response state.

Which interface element should become a reusable Flutter widget/component first, and which screens will use it?

The minimal primary action button styled with our blue gradient visual theme. It handles loading states automatically and will be used on the Login, Register, and Classification screens.

What was the most important change your team made between narrow/mobile, medium/tablet, and wide/desktop layouts?

Moving from a mobile bottom navigation bar to a persistent left-hand Navigation Rail on desktop viewports. This makes better use of wide horizontal space and keeps the clean, minimal aesthetic intact on large monitors.

Which validation, loading, empty, success, or error state would create the greatest usability problem if it were not designed before implementation?

The error state for the MobileNetV3 classification API. If the ML inference times out and there is no explicit error state or retry button, the user might assume the application froze and close the browser, losing their image.

What specific design issue must your team resolve before beginning the next Flutter client-side implementation activity?

We need to finalize the Draw.io UI flow diagrams to explicitly map out the routing guards for expired JWT tokens, ensuring users are gracefully redirected to the login screen instead of encountering a blank page.

